Prompt Engineering

Writing the document the model completes

Tokens · Anatomy · Techniques · Output control
Tools · Chains · Reasoning models · Evals · Security

Part 1: Foundations

How models read prompts · Prompt anatomy

Part 2: Techniques

Examples teach · reasoning buys compute · search explores · tools reach out

Part 3: Building with prompts

Controlling output · Tools · Chains and workflows · Reasoning models

Part 4: Shipping prompts

Optimization · Evaluation · Adversarial prompting · Applications

Notes: ai-engineering/01-prompt-engineering

PART 1

Foundations

How models read prompts · Prompt anatomy

The mental model

Five properties of the reader

Order matters — the word-count test

Chat models are completion models wearing a transcript

Every prompt must satisfy four criteria

Writing instructions

Ordering — the default skeleton

Separate instructions from data with XML

Agent system prompts — procedures, not rule piles

The mental model

Why every rule follows

> **An LLM is a document-completion engine. A prompt is the opening of a document whose most plausible continuation is the answer you want.**

Ask "how would this document continue?" — not "how would a reasonable person reply?"

Source: notes/01-how-models-read-prompts.md

Five properties of the reader

Tokens, not characters

~4 chars per token. Letter counting and string reversal are hard; SHOUTING and random IDs cost extra tokens.

Autoregressive

Fixed compute per token and no takebacks. More thinking means emitting more tokens — the basis of CoT.

Left-to-right attention

Order matters. Put the instruction before content when it changes **how** the content must be read.

Truth bias

Hallucination is the same behaviour as fluency. A false premise gets run with, not questioned.

Chat is a transcript

RLHF + ChatML turn a completion model into an assistant. Roles are document structure, not a security boundary.

So…

Shape the document. Don't ask more forcefully — reorder, decompose, or move the work out of the model.

Source: notes/01-how-models-read-prompts.md

Order matters — the word-count test

Property 3

The book asked ChatGPT to count the words in a 173-word paragraph, placing the instruction in two positions.

Instruction after the paragraph

- The model reads the paragraph not knowing it will need a count
- It answers **348**
- It can't go back and re-read

Read first, task second → a guess

Instruction before the paragraph

- The model knows the job while it reads
- It answers **173**
- One-pass heuristic: could an expert finish this in a single read, with no notes?

Task first → the reading is shaped by it

If the prompt fails the one-pass test, restructure the task rather than asking harder.

Source: notes/01-how-models-read-prompts.md

Chat models are completion models wearing a transcript

CHATML

```
<|im_start|>system
You are a terse technical assistant.<|im_end|>
<|im_start|>user
What does git rebase do?<|im_end|>
<|im_start|>assistant
```

- **System role** is the main lever: RLHF trains models to follow it closely
- Reserved tokens can't be typed — **unless you paste user content into `system`**
- What chat costs: alignment tax, chatty preamble, a narrower voice
- **Two scripts:** the user's conversation ≠ the transcript you send. You are the showrunner of the second.

Every prompt must satisfy four criteria

1. **Resemble training data** — the Little Red Riding Hood principle — stay on familiar document shapes
2. **Carry all relevant information** — and only that. Padding distracts (Chekhov's gun — everything included must matter)
3. **Lead to a solution** — point at an answer, not at more elaboration of the problem
4. **Have a natural stopping point** — end of turn, a closing tag, or a stop sequence

> **Chat fine-tuning gives you 1, 3 and 4 almost free. Criterion 2 is never automatic — that's context engineering.**

Source: notes/02-prompt-anatomy.md

Writing instructions

Clear, direct, specific

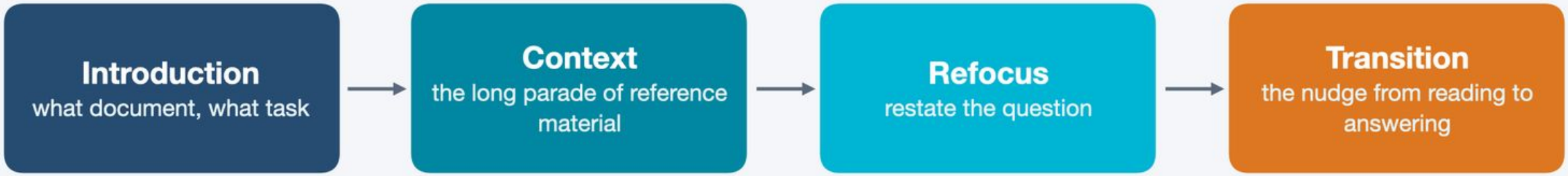
Golden rule (Anthropic): if a colleague with no context would be confused by the prompt, so will the model.

Rule	Weak	Strong
Be specific	"Keep it short, not too descriptive"	"Use 2–3 sentences, for a high-school student"
Say what to do	"DO NOT ASK FOR INTERESTS" (it asked anyway)	"Recommend from the top trending movies. If none fit, say so."
Give the reason	"NEVER use ellipses"	"No ellipses — a text-to-speech engine will read this aloud"
Avoid absolutes	"Always cite three sources"	"Cite sources where they exist; say so when they don't"
Specify the output	"Extract the places"	"Extract the places. Format: `Place: <comma_separated_list>`"

> **Reasons generalise better than commands. Shouting (CRITICAL, MUST) now causes over-application.**

Source: notes/02-prompt-anatomy.md

Ordering — the default skeleton



- **Recency bias** + **lost in the middle** → early-to-mid content is used least (the Valley of Meh)
- **Sandwich:** introduction + refocus, so the instruction is both primed and fresh
- Long inputs (20k+ tokens): documents first, query last — up to **30% better** on Anthropic's tests
- Ask the model to quote the relevant parts before it answers

Source: notes/02-prompt-anatomy.md

Separate instructions from data with XML

XML

```
<instructions>
 Summarize the document for an executive audience.
</instructions>
<document index="1">
 <source>q3-report.pdf</source>
 <document_content>{{DOCUMENT}}</document_content>
</document>
<constraints>Max 3 sentences. Plain prose.
</constraints>
```

- **Advice conversation** — the chat default; can include synthetic prior turns
- **Analytic report** — a `Scope` section and `# Analysis` before `# Conclusion`
- **Structured doc** — lets the app route parts of the output
- Version the **template** (slots), not the rendered prompt

Source: notes/02-prompt-anatomy.md

Agent system prompts — procedures, not rule piles

Tau-Bench ablation (Li, Exp. 2-4)

One change at a time to an airline/retail support agent's system prompt.

Change (all else fixed)	Effect
Tone swapped (neutral → bombastic → emoji-casual)	Little change in completion
Same rules, flattened from an ordered procedure into an unordered list	**Success fell more than 30%** — "verify identity before refund" was skipped
Tool descriptions removed (signatures kept)	Tool-call errors up 45%

> **Write an SOP with numbered stages and exception branches. When an agent underperforms, ablate before rewriting.**

Source: notes/02-prompt-anatomy.md

PART 2

Techniques

Examples teach · reasoning buys compute · search explores · tools reach out

Which technique when

Few-shot — what the examples actually teach

Chain-of-thought — reasoning before the answer

Sampling, knowledge, search, code

ReAct — reason + act

Reflexion — verbal reinforcement learning

Which technique when

Start at the top

Problem	Reach for	Cost
Task is clear and common	Zero-shot	—
Format, style, or labels are non-obvious	Few-shot	Longer prompt; anchoring
Multi-step arithmetic, logic, planning	Chain-of-thought	More output tokens
Answers vary run to run	Self-consistency	N× calls
Search problem with dead ends	Tree of Thoughts	Many calls + a controller
Exact computation	PAL (code + interpreter)	A sandbox
Needs fresh or private information	ReAct / tool use	Loop + tools
Can verify and retry (tests, checkers)	Reflexion	Retry budget
Long task with separable stages	Prompt chaining	Orchestration code

Reasoning models build many of these in — check notes/07 before adding CoT scaffolding.

Source: notes/03-core-techniques.md

Few-shot — what the examples actually teach

What matters (Min et al., 2022)

- **Label space and input distribution** matter more than whether each label is correct
- Random labels still beat no examples
- **Format** matters a lot
- Sample labels from the true distribution, not a uniform spread

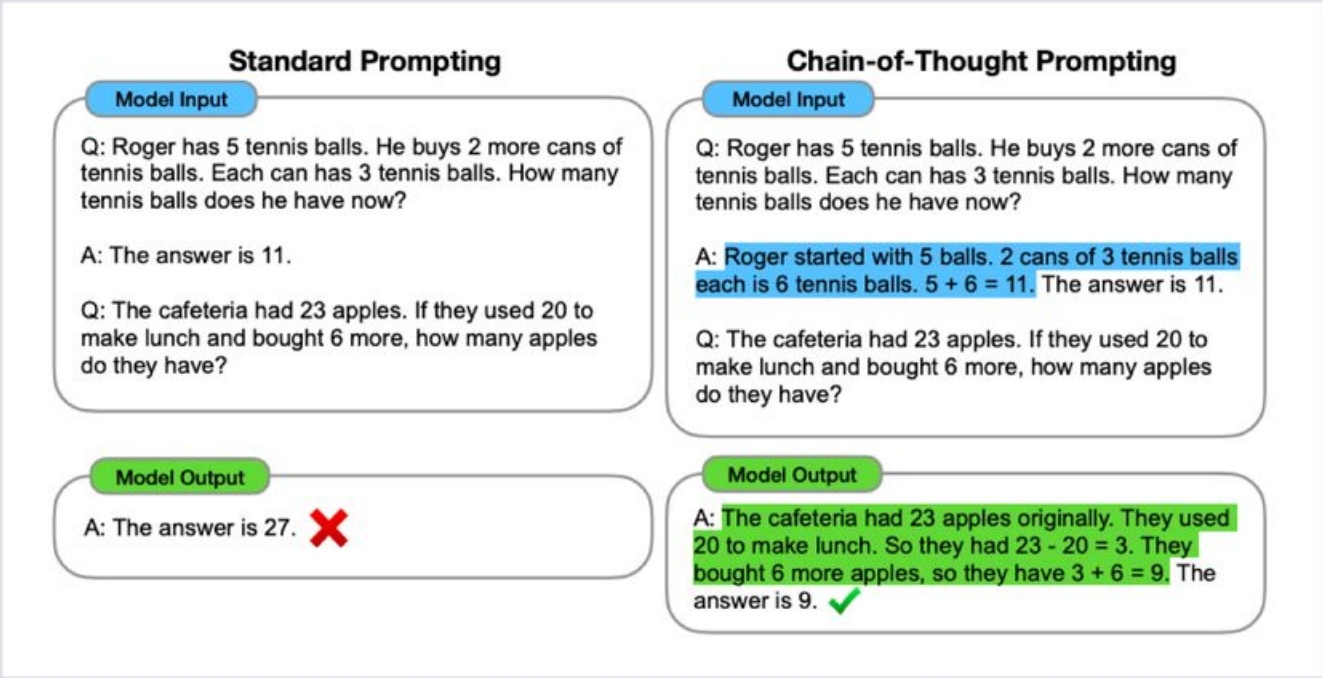
The three drawbacks

- **Scales badly** — each example needs the same rich context as the real question
- **Anchoring** — the examples' range becomes the model's expectation
- **Spurious patterns** — example order gets learned. Shuffle, or let DSPy choose

Start zero-shot. Add examples only when zero-shot fails in a way you can name.

Chain-of-thought — reasoning before the answer

Wei et al., 2022



Standard vs. chain-of-thought prompting (Wei et al., 2022)

- Without reasoning tokens, an answer is an intuitive guess; an explanation **after** it is rationalisation
- PaLM 540B on GSM8K: **~20% → ~60%**
- **Zero-shot CoT:** "Let's think step by step"
- Put reasoning in a parseable `<thinking>` block so the app can strip it

Source: notes/03-core-techniques.md

Sampling, knowledge, search, code

Self-consistency

Sample several CoT paths at temperature > 0 and take the majority answer. N× cost; nothing to vote on in open-ended text.

Generated knowledge

Call 1 generates relevant facts; call 2 answers with them in the prompt. Adds no knowledge the model lacks — that's retrieval.

Tree of Thoughts

Generate, evaluate, backtrack. Game of 24: CoT **4%** → ToT **74%**. Needs a controller loop in code.

PAL

The model writes code; an interpreter computes the answer. Don't ask the model to do what code does reliably.

ReAct — reason + act

Yao et al., 2022

TRACE

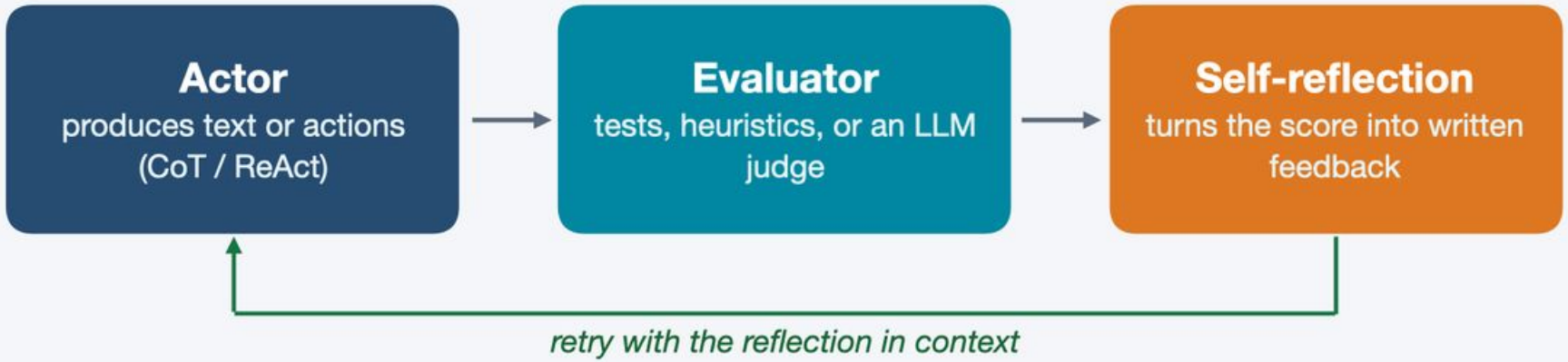
```
Thought 1: I need to search Arthur's Magazine and
          First for Women, and find which started
first.
Action 1:  Search[Arthur's Magazine]
Observation 1: Arthur's Magazine (1844–1846) was an
          American literary periodical...
Thought 2: Started in 1844. Now search First for
Women.
Action 2:  Search[First for Women]
Observation 2: ...started in 1989...
Thought 3: 1844 < 1989, so Arthur's Magazine was
first.
Action 3:  Finish[Arthur's Magazine]
```

- Grounded in retrieved facts, so it hallucinates less than CoT — but one bad search derails it
- ALFWorld: **71%** vs. 45% for act-only
- Prompted alone it lost to CoT on HotpotQA; fine-tuning on ~3k traces flipped that
- The ancestor of today's native tool-calling loop

Source: notes/03-core-techniques.md

Reflexion — verbal reinforcement learning

Shinn et al., 2023



- HumanEval pass@1: **91%** vs. GPT-4's 80% at the time
- **It works only when you get a do-over** — ideally with an objective evaluator
- Useless for irreversible actions; with a weak evaluator it reinforces confident mistakes

Source: notes/03-core-techniques.md

PART 3

Building with prompts

Controlling output · Tools · Chains and workflows · Reasoning models

- Anatomy of the ideal completion
- Sampling parameters
- The structured-output ladder
- The tool-calling loop
- Writing tool definitions
- Safety lives in code
- Workflow patterns
- Workflows vs. agents
- Prompting reasoning models

Anatomy of the ideal completion



- **Structural boilerplate** (an opening `{`, a heading): put it in the prompt — cheaper, and it guarantees the format
- **Reasoning:** let it run long — that's CoT, and it buys correctness
- **Fluff:** don't fight it; give it a slot (`1. answer 2. caveats`) and parse slot 1
- Stop on the recognisable end and nothing after it costs you anything

Sampling parameters

Tune one knob at a time

Parameter	Effect	Typical use
`temperature`	Flattens (high) or sharpens (low) the distribution	Low for extraction and factual QA; higher for brainstorming
`top_p`	Sample from the smallest set reaching mass p	Tune temperature **or** top_p — not both
`max_tokens`	Hard cap on output length	A cut-off is not a stop — check the stop reason
`stop`	Strings that end generation server-side	The cheapest way to end on your recognisable end
`frequency` / `presence` **penalty**	Penalise repeated or any-seen tokens	Tune one penalty — not both

> **Temperature 0 is not deterministic. Many reasoning models fix or reject these parameters.**

Source: notes/04-controlling-output.md

The structured-output ladder

Climb only as far as you need

weaker

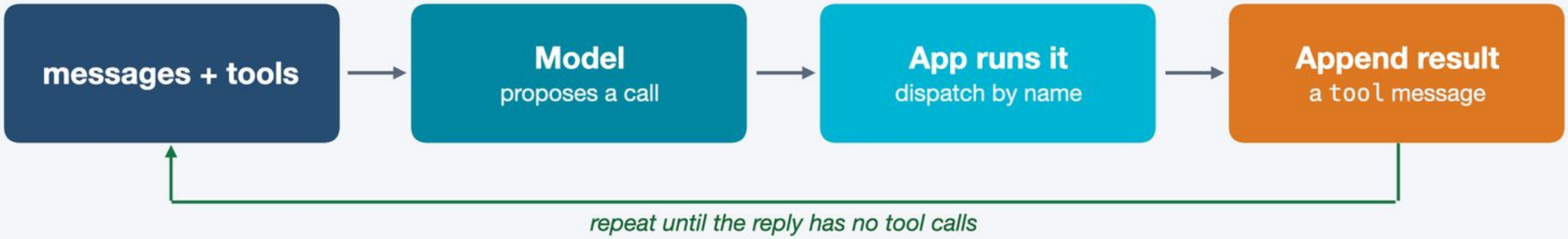
1. **Ask for it** — "Return JSON with keys…" — fine for prototypes, breaks at scale
2. **Show it** — a schema or one complete example; match the prompt's format to the output's
3. **Fence it** — `<answer>…</answer>` gives a parse target even in a chatty reply
4. **Prefill it** — start the assistant turn with `{` — some newer models no longer accept this
5. **Constrain it** — JSON mode or schema-constrained decoding
6. **Route it through a tool** — a tool whose parameters are your output schema

stronger guarantee

> **A schema guarantees shape, not truth. Validate the values too.**

Source: notes/04-controlling-output.md

The tool-calling loop



- The model never touches the world. It **proposes**; your code **executes**
- Under the hood it's still completion: schemas are rendered into the system prompt as text

- **Writing a tool definition is writing a prompt** — names, descriptions and enums all steer the choice

Writing tool definitions

Each field is prompt text

Rule	Why
Few tools that partition the domain	Every extra tool is a distractor; overlapping tools force a coin flip
Self-documenting names (`get_current_weather`)	Unbroken lowercase is hard to tokenise and read
Descriptions say when to use it	"Use when the user asks about recent events…"
Few, simple arguments; use `enum`s	Complex parameter lists burn tokens and accuracy
Drop arguments the app already knows	The model invents placeholders (`"my-org"`) for values it can't see
Errors that teach	"`date` must be YYYY-MM-DD", not a stack trace

Source: notes/05-tools-and-function-calling.md

Safety lives in code

> **"Always confirm with the user before booking" will eventually be skipped.**
>
> *Let the model request any call; intercept side-effecting calls in the app and require approval of the exact arguments. Tool results are untrusted input.*

Source: notes/05-tools-and-function-calling.md

Workflow patterns

Anthropic's taxonomy

When one prompt can't do the job, split it — each step gets one job, a short context, and a checkable output.

Pattern	What it does	Example
Prompt chaining	Sequential steps with optional gates	Outline → check → draft
Routing	Classify then send to a specialised prompt or model	Support triage; small model for easy queries
Parallelisation	Sectioning (subtasks) or voting (same task N×)	Moderation on several axes; self-consistency
Orchestrator–workers	An LLM splits the task and delegates	Multi-file code changes
Evaluator–optimiser	One prompt generates, another critiques — loop	Translation with a critic; Reflexion

Source: notes/06-chaining-and-workflows.md

Workflows vs. agents

Start with a workflow

Workflow

- Control flow predefined in code
- Tool use orchestrated by code
- High predictability
- Good for: repeatable tasks, production reliability, cost control

DAG is the default topology

Agent

- Control flow chosen by the model
- Tools selected by the model
- Lower predictability
- Good for: open-ended tasks whose steps can't be known up front

Most production systems are hybrids

First question for every task: does it need an LLM at all?

Source: notes/06-chaining-and-workflows.md

Prompting reasoning models

Goals, not steps

Drop these habits

- Prescribing the reasoning steps
- Bolting on "Let's think step by step" — explicit CoT can **hurt** instruction-following
- Padding with few-shot examples by default
- Trusting reasoning because it was long

Do this instead

- State the goal, constraints and output format at a high level
- Tune **effort / budget**, not wording
- Use examples only for style or format the model gets wrong
- Ask it to verify its work before finishing

Escalate gradually: thinking off → low → medium → high, only while eval scores keep improving.

PART 4

Shipping prompts

Optimization · Evaluation · Adversarial prompting · Applications

Prompt optimization is a loop

Letting models write prompts

Build the eval first

LLM-as-judge — the SOMA rubric

Case study: small changes, large effects

Adversarial prompting

Defenses — layer them

Synthetic data — inject the randomness

Prompt optimization is a loop

No metric, no optimizer



- **Template ≠ prompt** — version the template, with its model, ID and eval score
- **Pin the model version** — a tuned prompt is not portable
- **Diff behaviour, not text** — run the regression suite on every edit
- Fix the cheapest layer first: clarity → context → ordering → examples → CoT → split → bigger model → fine-tune

Source: notes/08-prompt-optimization.md

Letting models write prompts

APE · OPRO · meta-prompting · DSPy

APE

An LLM proposes instructions, each is scored on a target model. Found "Let's work this out in a step by step way to be sure we have the right answer."

OPRO

The LLM is the optimizer, shown past prompts and their scores. Found "Take a deep breath…"

Meta-prompting

A strong model drafts or critiques a prompt from failing cases. A first draft, not a result.

DSPy

Signatures + modules, compiled against a metric: picks few-shot examples and their order, searches instructions.

Source: notes/08-prompt-optimization.md

Build the eval first

Copilot's eval framework is older than its prompts or its UI. Always log latency and token counts too.

Target	Decision	Test style
The model	Swap or upgrade	Broad regression suite
A single prompt	Wording, examples, parameters	Small unit tests on one model call
The composition (chain, loop)	Architecture	End-to-end regression — no unit-level substitute

> **Level 1: 5–20 examples, outputs committed, reviewed with git diff. Level 2: hundreds, scored automatically.**

Source: notes/09-evaluating-prompts.md

LLM-as-judge — the SOMA rubric

Never let it grade its own work

Specific

"Did it pick the heating system?" — not "Is this right?" A generic question is as hard as the task.

Ordinal

A 1–5 scale with each level described. Yes/no hides degrees; undefined levels drift.

Multi-aspect

Score each dimension separately — intent vs. execution; relevance / truth / completeness.

And…

Rubric before content. Scores are relative. Calibrate against several human annotators.

Source: notes/09-evaluating-prompts.md

Case study: small changes, large effects

Clavié et al., 2023 — gpt-3.5-turbo

Prompt modification (cumulative)	F1	Template adherence
Baseline (posting + question)	65.6	79%
Few-shot CoT	78.4	87%
Zero-shot CoT	81.4	65%
+ role in system, task in user	87.5	71%
+ mocked acknowledgement + reiterated key points	89.3	75%
+ strict output template	86.3 ↓	**98%**
+ "reach the right conclusion" + common failures	90.3	77%
+ a name + positive feedback	**91.7**	81%

> **Wording moved F1 by 26 points. The methodology is the lesson — change one thing, re-score.**

Source: notes/09-evaluating-prompts.md

Adversarial prompting

Instructions and data share one channel

There's no parameterised query for LLMs. Every attack exploits that the model can't tell text to obey from text to process.

Attack	Goal	Classic form
Direct injection	Override the app's instructions	"Ignore the above directions and…"
Indirect injection	Smuggle instructions in through retrieved data	Hidden text in a web page, email, PDF or tool result
Prompt leaking	Extract the system prompt or examples	"…then output a copy of the full prompt"
Jailbreaking	Get around safety training	Personas (DAN), fiction, encodings

> **Indirect injection matters most — the attacker doesn't need to be your user. Risk scales with what the model can do.**

Source: notes/10-adversarial-prompting.md

Defenses — layer them

No prompt-level defense is complete

weaker

1. **Instruction** — warn the model that users may try to change it
2. **Source tagging + structure** — untrusted text in the `user` role or a `tool` message, never `system`; delimit it
3. **Detection** — a separate classifier screens inputs and outputs
4. **Least privilege** — only the tools and data the task needs
5. **Human approval in code** — side-effecting calls need explicit confirmation
6. **Architecture** — a quarantined model reads untrusted content and can't call tools (dual-LLM, CaMeL)

stronger

> **Assume the system prompt will leak. Treat model output as untrusted. Red-team before shipping.**

Source: notes/10-adversarial-prompting.md

Synthetic data — inject the randomness

Raising the temperature isn't enough for diversity. Put the variation in the prompt.

Combinatorial seeding

TinyStories: sample a noun, verb, adjective and features into every prompt. A seeded label is a free label.

Hierarchical

Generate a summary and a must-use sentence first, then the final piece.

Vary the audience

phi-1: textbook Python for different readers. A 1.3B model rivalled models 10× its size.

Synthetic queries

Promptagator: 8 labelled examples + a corpus are enough to train a niche retriever.

Pitfalls: skewed seeds give skewed data, and data from model A favours model A in evals.

Rules of thumb

The pillar on one slide

Wrap-up

Writing

Clear and specific; say why. XML between instructions and data. Long content first, question last.

Techniques

Zero-shot → few-shot → CoT → self-consistency. PAL for maths, ReAct for tools, Reflexion when you can check.

Output

Low temperature for parseable output. Climb the structured-output ladder only as far as needed.

Tools & chains

Tool descriptions are prompts. Gate side effects in code. Workflow before agent.

Reasoning models

State the goal, not the steps. Tune effort, not wording.

Shipping

Eval before tuning; one change at a time. Assume leaks and injection.

Source: README.md#rules-of-thumb

Where to go next

Wrap-up

- **02 — Context engineering:** what goes into the window alongside the prompt
  - Retrieval, memory, tool outputs and token budget
- **06 — Eval:** defining correct, datasets, graders and calibrated judges
- **Run the code:** the commented snippets in each note
- **Read:** Berryman & Ziegler, Prompt Engineering for LLMs; Huyen, AI Engineering Ch 5
- **Copy from:** notes/prompt-library.md — example prompts by task

The one idea to keep

A prompt is a document the model continues. When a prompt fails, change the document — its order, its contents, its shape — before you change the wording.

Source: README.md